The PostgresChecker: Hunting Use-after-free Bugs in PostgreSQL

Bernd Reiß

April 28, 2026

Outline

1. Problem Analysis: PostgreSQL & C
2. Static Code Analysis Tools
3. Analysis of the PG Code Base
4. Results and Experimental Evaluation

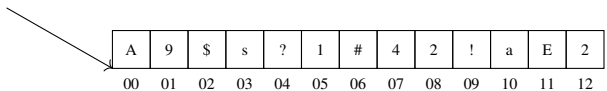
Problem Analysis: PostgreSQL & C

Memory Safety in C: Hello Memory


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  printf("%s", str);

  free(str);

  return 0;

}
```

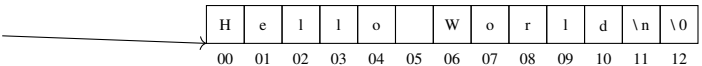
Memory Safety in C: Hello Memory


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   printf("%s", str);
12
13   free(str);
14
15   return 0;
16
17 }
```



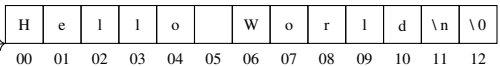
Memory Safety in C: Hello Memory


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   printf("%s", str);
12
13   free(str);
14
15   return 0;
16
17 }
```



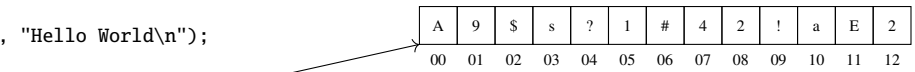
Memory Safety in C: Hello Memory


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   printf("%s", str);
12
13   free(str);
14
15   return 0;
16
17 }
```



Memory Safety in C: Hello Memory


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   printf("%s", str);
12
13   free(str);
14
15   return 0;
16
17 }
```



Memory Safety in C: Hello Memory


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  printf("%s", str);

  free(str);

  return 0;

}
```

```
$ clang hello.c -o hello
```

Memory Safety in C: Hello Memory


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  printf("%s", str);

  free(str);

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
```

Memory Safety in C: Hello Memory


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  printf("%s", str);

  free(str);

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
Hello World
$
```

Memory Safety in C: Hello Memory


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  printf("%s", str);

  free(str);

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
Hello World
$
```

Memory Safety in C: Use-After-Free


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   free(str); <----------------------
12
13   printf("%s", str); <--------------
14
15   return 0;
16
17 }
```

Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str); <----------------------

  printf("%s", str); <--------------

  return 0;

}
```

```
$ clang hello.c -o hello
```

Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str); <-----------------------

  printf("%s", str); <--------------

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
```

Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str); <-----------------------

  printf("%s", str); <--------------

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
|~W$
```

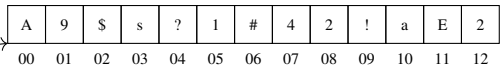
Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str); <-----------------------

  printf("%s", str); <--------------

  return 0;

}
```

```
$ clang hello.c -o hello
$ ./hello
|~W$
```

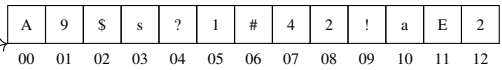
Memory Safety in C: Use-After-Free


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   free(str);
12
13   printf("%s", str);
14
15   return 0;
16
17 }
```



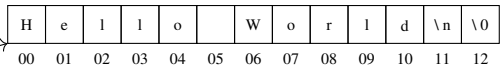
Memory Safety in C: Use-After-Free


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   free(str);
12
13   printf("%s", str);
14
15   return 0;
16
17 }
```



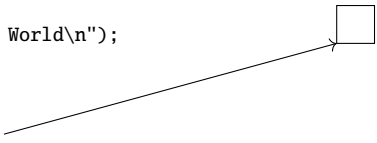
Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str);

  printf("%s", str);

  return 0;

}
```



Memory Safety in C: Use-After-Free


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    strcpy(str, "Hello World\n");
10
11   free(str);
12
13   printf("%s", str);
14
15   return 0;
16
17 }
```



Memory Safety in C: Use-After-Free


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str);

  printf("%s", str);

  return 0;

}
```

Potential issues [Oor23]:

- Garbage values
- Corrupted data
- Security issues
- Program crashes

PostgreSQL-Style Functions

Function for SQL integration in C [Pos25d]:

```
1  void
2  output_simple_statement(char *stmt,
3                          int whenever_mode)
4  {
5    output_escaped_str(stmt, false);
6    if (whenever_mode)
7      whenever_action(whenever_mode);
8    output_line_number();
9    free(stmt); <----------------------
10 }
```

PostgreSQL-Style Functions

Function for SQL integration in C [Pos25d]:

```c
void
output_simple_statement(char *stmt,
                        int whenever_mode)
{
  output_escaped_str(stmt, false);
  if (whenever_mode)
    whenever_action(whenever_mode);
  output_line_number();
  free(stmt); <----------------------
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 24);
  strcpy(str, "Hello World\n");

  output_simple_statement(str, 0);<-Freed!!!

  printf("%s", str); <------ Use-after-free

  return 0;
}
```

Static Code Analysis

Why bother?

Definition: Static program analysis = analyzing a program without running it

Why bother?

Definition: Static program analysis = analyzing a program without running it

Several use cases:

Why bother?

Definition: Static program analysis = analyzing a program without running it

Several use cases:

- Compiler basics:
  - Syntactic checks
  - Type checking

Why bother?

Definition: Static program analysis = analyzing a program without running it

Several use cases:

- Compiler basics:
  - Syntactic checks
  - Type checking
- Compiler optimizing code:
  - Eliminating dead or redundant code
  - Optimize register allocation

Why bother?

Definition: Static program analysis = analyzing a program without running it

Several use cases:

- Compiler basics:
  - Syntactic checks
  - Type checking
- Compiler optimizing code:
  - Eliminating dead or redundant code
  - Optimize register allocation
- Beyond the compiler:
  - Use-after-free checks
  - Null pointer dereference

Tools: Clang

- Full-fledged compiler
- Static Analyzer (e.g., MallocChecker)
- Easily extendible
- [Cla25a; Cla25b; Mur24]

Tools: Clang

- Full-fledged compiler
- Static Analyzer (e.g., MallocChecker)
- Easily extendible
- [Cla25a; Cla25b; Mur24]

```
$ clang --analyze hello.c
hello.c:13:3:  warning:  Use of memory after it is freed
[unix.Malloc]
13 | printf("%s", str);
   | ^~~~~~~~~~~~~~~~~
1 warning generated.
$
```

Tools: Clang

- Full-fledged compiler
- Static Analyzer (e.g., MallocChecker)
- Easily extendible
- [Cla25a; Cla25b; Mur24]

```
$ clang --analyze hello.c
hello.c:13:3:  warning:  Use of memory after it is freed
[unix.Malloc]
13 | printf("%s", str);
   | ^~~~~~~~~~~~~~~~~
1 warning generated.
$
```

How to get functions from PG source code?

Tools: Coccinelle

- Pattern matching tool
- Able to perform *transformations*
- Uses *semantic patches*
- Print diagnostic via Python scripting
- [Bru09; Law18; Law22; Mul06; Ole10; Pad07]:

Tools: Coccinelle

- Pattern matching tool
- Able to perform *transformations*
- Uses *semantic patches*
- Print diagnostic via Python scripting
- [Bru09; Law18; Law22; Mul06; Ole10; Pad07]:

```
1 @forall@
2 type t;
3 identifier x;
4 @@
5 t x;
6 ...
7 -free(x);
8 +pfree(x);
```

Analysis of the PG Code Base

PostgreSQL Data Basis

- Find all functions in the PostgreSQL code base that use `free` or `realloc`
- Recurse with newly found functions
- Stop when no new functions are found

```
                pfree ->          ...
free ->       pg_free ->          ...
        output_statement ->       ...
```

PostgreSQL Data Basis

- Find all functions in the PostgreSQL code base that use `free` or `realloc`
- Recurse with newly found functions
- Stop when no new functions are found

```
                    pfree ->          ...
free ->          pg_free ->           ...
          output_statement ->         ...
```

One script to rule them all:

```
1  @match exists@
2  identifier f, i;
3  type rt, t;
4  position p;
5  @@
6  rt f(..., t i, ...) {
7    <+...
8    free@p(..., i, ...)
9    ...+>
10 }
11 @script:python@
12 f << match.f;
13 ...
14 @@
15 print(f">{f}, " + p[0].file + ":"...
16 ...
```

Category 1: Strict Functions

Functions that ALWAYS call free/realloc on a parameter

Category 1: Strict Functions

Functions that ALWAYS call free/realloc on a parameter

```
1  void
2  output_simple_statement(char *stmt, int whenever_mode)
3  {
4    output_escaped_str(stmt, false);
5    if (whenever_mode)
6      whenever_action(whenever_mode);
7    output_line_number();
8    free(stmt);
9  }
```

Category 2: Dependent Functions

Functions that call free/realloc DEPENDENT on a parameter

Category 2: Dependent Functions

Functions that call free/realloc DEPENDENT on a parameter

```
1  void
2  ExecForceStoreMinimalTuple(MinimalTuple mtup,
3                 TupleTableSlot *slot,
4                 bool shouldFree)
5  {
6    ...
7      if (shouldFree)
8      {
9        ExecMaterializeSlot(slot);
10       pfree(mtup);
11     }
12   }
13 }
```

[Pos25c]

Category 3: Arbitrary Functions

Functions that ARBITRARILY call free/realloc on a parameter

Category 3: Arbitrary Functions

Functions that ARBITRARILY call free/realloc on a parameter

```
1  void
2  add_path(RelOptInfo *parent_rel, Path *
       new_path)
3  {
4    bool accept_new = true;
5    ...
6    /*CHECKING WHETHER NEW PATH IS BETTER*/
7    ...
8    if (accept_new)
9    ...
10   else
11   {
12     ...
13     pfree(new_path);
14   }
15 }
```

[Pos25a]

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

0	1	2	3	4	5	6	7
0	1	1	0	1	0	0	1

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

0	1	1	0	1	0	0	1
0	1	2	3	4	5	6	7

```
1  Bitmapset *
2  bms_del_member(Bitmapset *a, int x)
3  {
4    ...
5    a->words[wordnum] &= ~((bitmapword) 1 <<
       bitnum);
6    ...
7      /* the set is now empty */
8      pfree(a);
9      return NULL;
10   ...
11   return a;
12 }
13 Intended usage: bms = bms_del_member(bms, 4);
```

[Pos25b]

Categories Overview

Category	Free	Realloc
All Functions	235	25
C1: Strict Functions	186	19
C2: Dependent Functions	14	1
C3: Arbitrary Functions	35	5
C4: Functions to Reassign	18	25
C5: Functions Returning Boolean	1	0

Results and Experimental Evaluation

Experimental Evaluation (1)

Repository	Files Analyzed	False Positives	True Positives
PostgreSQL src	1,059	2%	2
PostgreSQL contrib	138	2%	0

Experimental Evaluation (2)

- PostgreSQL source code: 2 bugs -> 1 commited bug fix [Rei25]
- Citus: 1 bug -> 1 commited bug fix [Rei26a]
- Postgis: 1 bug -> 1 commited bug fix [Rei26b]

Confirmed Bug Resulting in Patch


```
1  static void
2  expand_partitioned_rtentry(..., RelOptInfo *relinfo,...)
3  {
4    Bitmapset  *live_parts;
5    ...
6    relinfo->live_parts = live_parts = prune_append_rel_partitions(relinfo);
7    ...
8    i = -1;
9    while ((i = bms_next_member(live_parts, i)) >= 0)
10   {
11     ...
12     childrel = try_table_open(childOID, lockmode);
13     if (childrel == NULL)
14     {
15       relinfo->live_parts = bms_del_member(relinfo->live_parts, i);
16       continue;
17     }
```

Limitations

Limitations

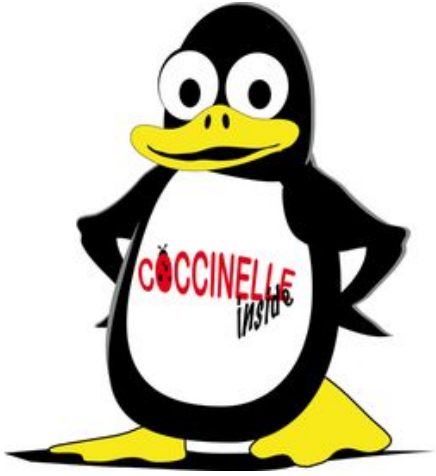
- Counter-like structures: e.g., `list->length`

Limitations

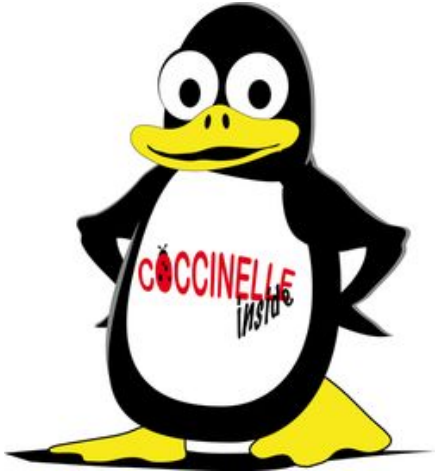
- Counter-like structures: e.g., `list->length`
- Error handling: certain levels abort execution

Limitations

- Counter-like structures: e.g., `list->length`
- Error handling: certain levels abort execution
- Control-flow-related issues: e.g., `if (ecpg_realloc)`



PostgresChecker-Demo + Bachelor Thesis + Presentation:
`https://github.com/berndreiss/pg_ladybug`



PostgresChecker-Demo + Bachelor Thesis + Presentation:
`https://github.com/berndreiss/pg_ladybug`
`https://berndreiss.net/postgreschecker`

References

References I

[Bru09] Julien Brunel et al. "A foundation for flow-based program matching: using temporal logic and model checking". In: *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '09. Savannah, GA, USA: Association for Computing Machinery, 2009, pp. 114–126. ISBN: 9781605583792. DOI: 10.1145/1480881.1480897. URL: https://doi.org/10.1145/1480881.1480897.

[Law18] Julia Lawall and Gilles Muller. "Coccinelle: 10 years of automated evolution in the Linux kernel". In: *Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference*. USENIX ATC '18. Boston, MA, USA: USENIX Association, 2018, pp. 601–613. ISBN: 9781931971447.

[Law22] Julia Lawall and Gilles Muller. "Automating Program Transformation with Coccinelle". In: *NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings*. Pasadena, CA, USA: Springer-Verlag, 2022, pp. 71–87. ISBN: 978-3-031-06772-3. DOI: 10.1007/978-3-031-06773-0_4. URL: https://doi.org/10.1007/978-3-031-06773-0_4.

[Mul06] Gilles Muller et al. "Semantic patches considered helpful". In: *SIGOPS Oper. Syst. Rev.* 40.3 (July 2006), pp. 90–92. ISSN: 0163-5980. DOI: 10.1145/1151374.1151392. URL: https://doi.org/10.1145/1151374.1151392.

[Mur24] Ivan Murashko. *Clang Compiler Frontend: Get to grips with the internals of a C/C++ compiler frontend and create your own tools*. Birmingham: Packt, 2024. ISBN: 978-1-83763-098-1.

References II

[Ole10] Mads Olesen et al. "Clang and Coccinelle: Synergising program analysis tools for CERT C Secure Coding Standard certification". In: *Electronic Communications of the EASST* 33 (Dec. 2010). DOI: 10.14279/tuj.eceasst.33.455. URL: https://eceasst.org/index.php/eceasst/article/view/1725.

[Oor23] Paul C. van Oorschot and Frank Piessens. "Memory Errors and Memory Safety: C as a Case Study". In: *IEEE Security and Privacy* 21.2 (Mar. 2023), pp. 70–76. ISSN: 1540-7993. DOI: 10.1109/MSEC.2023.3236542. URL: https://doi.org/10.1109/MSEC.2023.3236542.

[Pad07] Yoann Padioleau, Julia L. Lawall, and Gilles Muller. "SmPL: A Domain-Specific Language for Specifying Collateral Evolutions in Linux Device Drivers". In: *Electronic Notes in Theoretical Computer Science* 166 (2007). Proceedings of the ERCIM Working Group on Software Evolution (2006), pp. 47–62. ISSN: 1571-0661. DOI: https://doi.org/10.1016/j.entcs.2006.07.022. URL: https://www.sciencedirect.com/science/article/pii/S1571066106005287.

[Pos25a] PostgreSQL Global Development Group. *add_path*. 2025. URL: https://github.com/postgres/postgres/blob/6d6480066c1a96c7130b97b1139fdada9d484f80/src/backend/optimizer/util/pathnode.c#L461 (visited on 06/15/2025).

References III

[Pos25b] PostgreSQL Global Development Group. *bms_del_member*. 2025. URL: https://github.com/postgres/postgres/blob/6d6480066c1a96c7130b97b1139fdada9d484f80/src/backend/nodes/bitmapset.c#L868 (visited on 06/15/2025).

[Pos25c] PostgreSQL Global Development Group. *ExecForceStoreMinimalTuple*. 2025. URL: https://github.com/postgres/postgres/blob/98749132e87dea1e79bdddc8c20527697a9992af/src/backend/executor/execTuples.c#L1598 (visited on 06/15/2025).

[Pos25d] PostgreSQL Global Development Group. *output_simple_statement*. 2025. URL: https://github.com/postgres/postgres/blob/98749132e87dea1e79bdddc8c20527697a9992af/src/interfaces/ecpg/preproc/output.c#L18 (visited on 06/03/2025).

[Rei25] Bernd Reiß. *Commit da9f9f7*. 2025. URL: https://github.com/postgres/postgres/commit/da9f9f75e5ce27a45878ffa262156d18f0046188 (visited on 08/29/2025).

[Rei26a] Bernd Reiß. *Citus: Commit 2b69f6a*. 2026. URL: https://github.com/citusdata/citus/pull/8477 (visited on 04/26/2026).

References IV

[Rei26b] Bernd Reiß. *Postgis: Commit 46bf883788.* 2026. URL: https://gitea.osgeo.org/postgis/postgis/pulls/281 (visited on 04/26/2026).

[Cla25a] The Clang Team. *Clang 21.0.0git documentation.* 2025. URL: https://clang.llvm.org/docs/index.html (visited on 06/10/2025).

[Cla25b] The Clang Team. *Extra Clang Tools 21.0.0git documentation.* 2025. URL: https://clang.llvm.org/extra (visited on 06/10/2025).

Results and Experimental Evaluation

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

- `ProgramState`: program environment (including variables)

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

- `ProgramState`: program environment (including variables)
- `CallEvent`: abstraction of function calls

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

- `ProgramState`: program environment (including variables)
- `CallEvent`: abstraction of function calls
- `MemRegion`: abstraction of memory regions

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

- `ProgramState`: program environment (including variables)
- `CallEvent`: abstraction of function calls
- `MemRegion`: abstraction of memory regions
- `Stmt`: represents a statement

Clang Static Analyzer Framework (1)

Full-fledged compiler that provides useful classes:

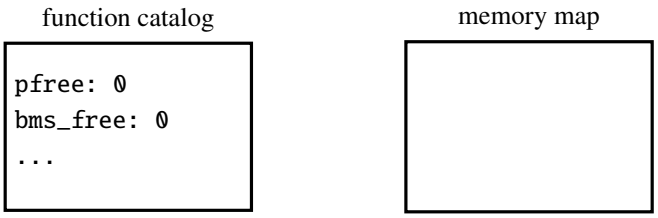
- `ProgramState`: program environment (including variables)
- `CallEvent`: abstraction of function calls
- `MemRegion`: abstraction of memory regions
- `Stmt`: represents a statement
- `Expr`: represents an expression

Clang Static Analyzer Framework (1)

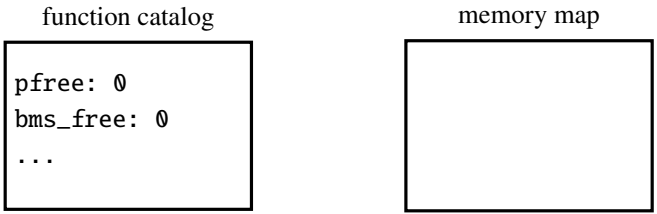
Full-fledged compiler that provides useful classes:

- `ProgramState`: program environment (including variables)
- `CallEvent`: abstraction of function calls
- `MemRegion`: abstraction of memory regions
- `Stmt`: represents a statement
- `Expr`: represents an expression
- `SVal`: symbolic value derived from symbolic execution

Clang Static Analyzer Framework (2)

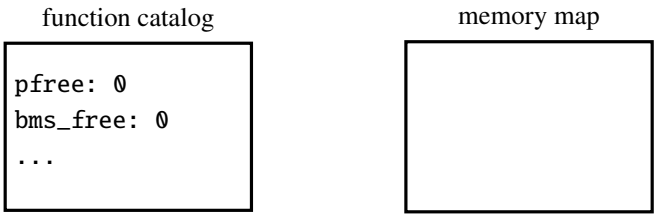


Clang Static Analyzer Framework (2)



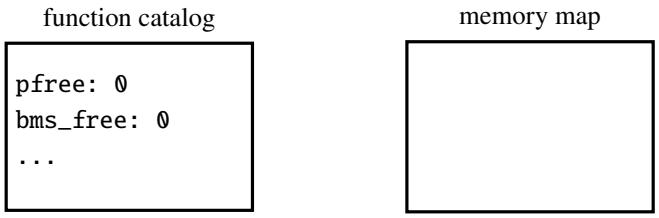

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

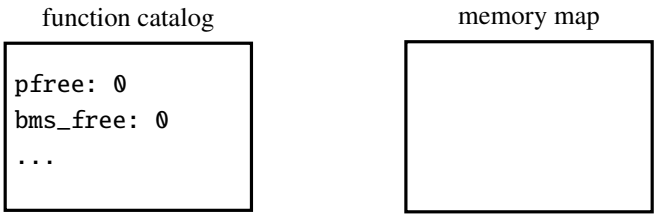
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address
0x42

Clang Static Analyzer Framework (2)

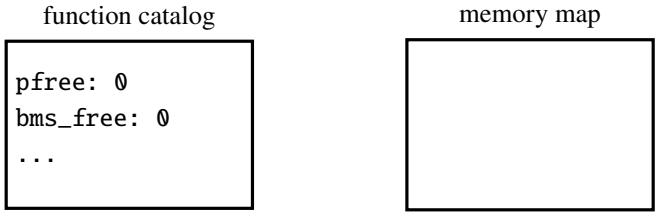



```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address

0x42

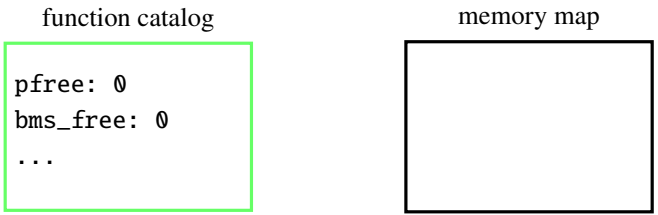
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

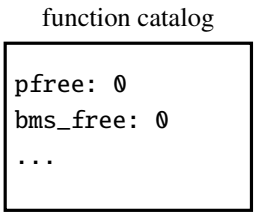
Clang Static Analyzer Framework (2)

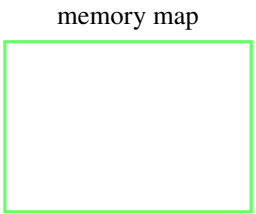



```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)

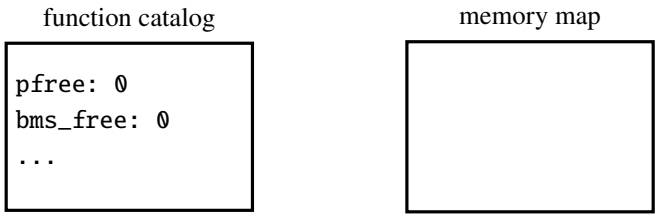





```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

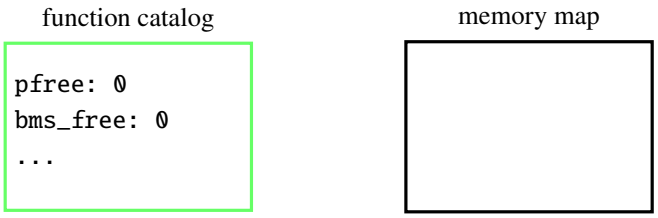
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

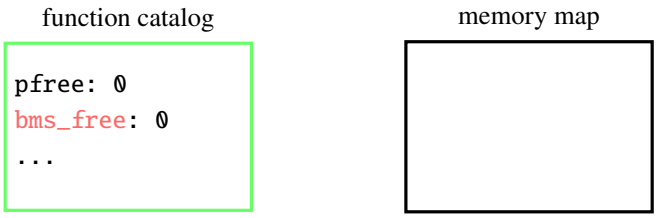
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

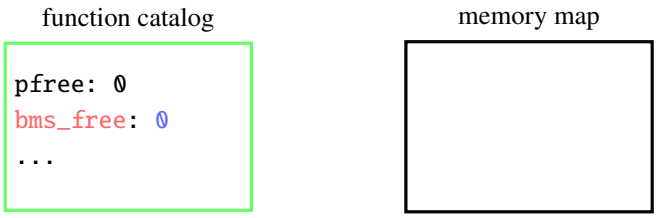
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

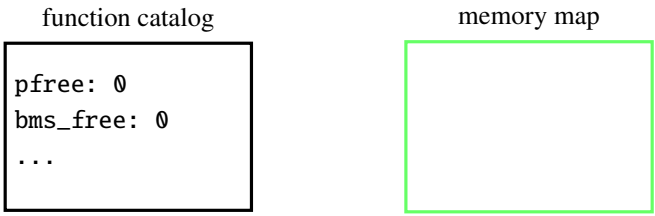
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

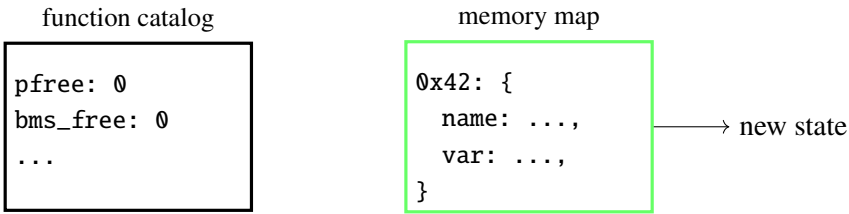
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

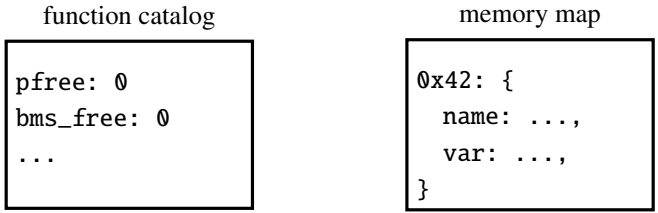
Clang Static Analyzer Framework (2)




```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)



Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address → 0x42

Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

Address

0x42

Clang Static Analyzer Framework (2)

function catalog

```
pfree: 0
bms_free: 0
...
```

memory map

```
0x42: {
  name: ...,
  var: ...,
}
```

```
Bitmapset *bms = bms_make_singleton(5);
...
if (!bms_is_member(bms, 42)){
  bms_free(bms);
  bms_free(bms);
```

DOUBLE FREE!

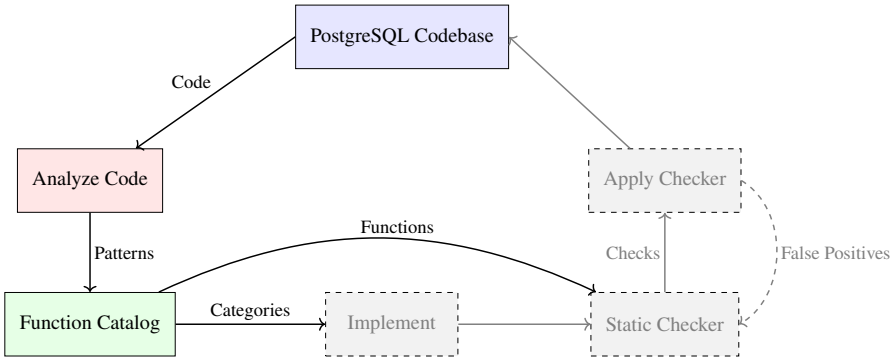
Clang Static Analyzer Framework (3)


```
test.c:68:3: warning: Attempt to free released memory: list1 [postgres.PostgresChecker]
   68 |   pfree(list1);
      |   ^~~~~~~~~~~~
test.c:67:3: note: Freeing function: pfree (list1)
   67 |   pfree(list1);
      |   ^
1 warning generated.
```

Clang Static Analyzer Framework (4)


```
1  {"ExecForceStoreMinimalTuple", DependencyInfo(0, false, [](CallEvent &Call,
2                                                            CheckerContext &C){
3    SVal shouldFree = Call.getArgSVal(2);
4    if (std::optional<nonloc::ConcreteInt> intVal =
5                                      shouldFree.getAs<nonloc::ConcreteInt>()) {
6      const llvm::APSInt &notFreeing = intVal->getValue();
7      if (notFreeing) return false;
8    }
9    return true;
10 })},
```

Results and Experimental Evaluation



Category 1: Strict Functions

Functions that ALWAYS call free/realloc on a parameter

Category 1: Strict Functions

Functions that ALWAYS call free/realloc on a parameter

```
1  void
2  output_simple_statement(char *stmt, int whenever_mode)
3  {
4    output_escaped_str(stmt, false);
5    if (whenever_mode)
6      whenever_action(whenever_mode);
7    output_line_number();
8    free(stmt);
9  }
```

Category 2: Dependent Functions

Functions that call free/realloc DEPENDENT on a parameter

Category 2: Dependent Functions

Functions that call free/realloc DEPENDENT on a parameter

```
1  void
2  ExecForceStoreMinimalTuple(MinimalTuple mtup,
3                 TupleTableSlot *slot,
4                 bool shouldFree)
5  {
6    ...
7      if (shouldFree)
8      {
9        ExecMaterializeSlot(slot);
10       pfree(mtup);
11     }
12   }
13 }
```

[Pos25c]

Category 3: Arbitrary Functions

Functions that ARBITRARILY call free/realloc on a parameter

Category 3: Arbitrary Functions

Functions that ARBITRARILY call free/realloc on a parameter

```
1  void
2  add_path(RelOptInfo *parent_rel, Path *
       new_path)
3  {
4    bool accept_new = true;
5    ...
6    /*CHECKING WHETHER NEW PATH IS BETTER*/
7    ...
8    if (accept_new)
9    ...
10   else
11   {
12     ...
13     pfree(new_path);
14   }
15 }
```

[Pos25a]

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

0	1	1	0	1	0	0	1
0	1	2	3	4	5	6	7

Category 4: Functions To Reassign

Functions that call free/realloc on a parameter where return value should be REASSIGNED

0	1	1	0	1	0	0	1
0	1	2	3	4	5	6	7

```
1  Bitmapset *
2  bms_del_member(Bitmapset *a, int x)
3  {
4    ...
5    a->words[wordnum] &= ~((bitmapword) 1 <<
       bitnum);
6    ...
7      /* the set is now empty */
8      pfree(a);
9      return NULL;
10   ...
11   return a;
12 }
13 Intended usage: bms = bms_del_member(bms, 4);
```

[Pos25b]

Category 5: Functions Returning Boolean

Functions that always return true/false when a parameter is freed/not freed

Category 5: Functions Returning Boolean

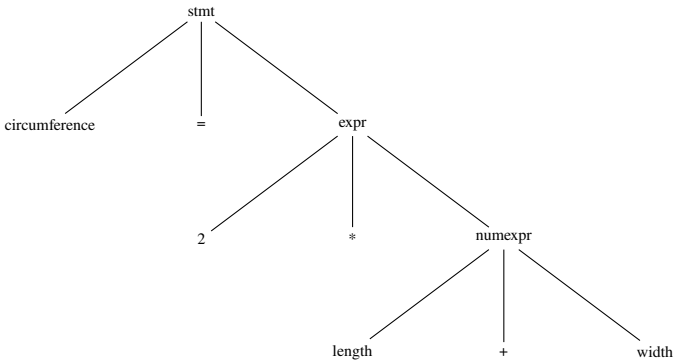
Functions that always return true/false when a parameter is freed/not freed

Compiler: Abstract Syntax Trees


```
float circumference = 2 * (length + width);
```

Compiler: Abstract Syntax Trees


```
float circumference = 2 * (length + width);
```

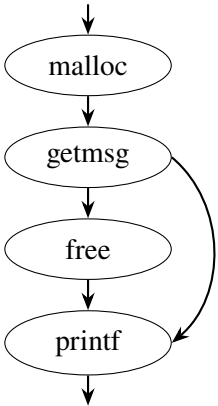


Compiler: Control-Flow-Graph


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```

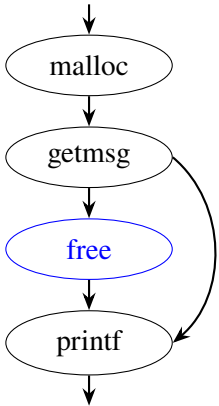
Compiler: Control-Flow-Graph -> flow-sensitive


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```



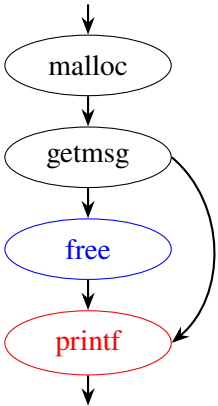
Compiler: Control-Flow-Graph -> flow-sensitive


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```



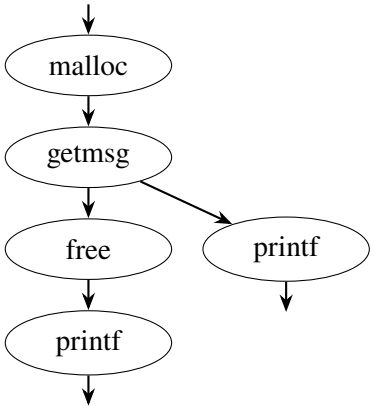
Compiler: Control-Flow-Graph -> flow-sensitive


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```



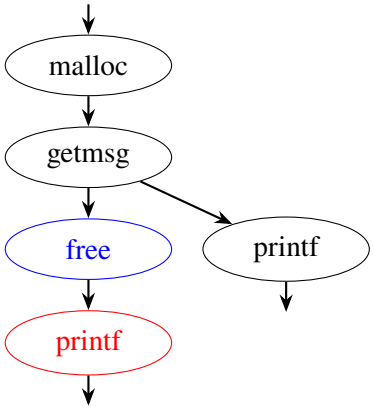
Compiler: Control-Flow-Graph -> path-sensitive


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```



Compiler: Control-Flow-Graph -> path-sensitive


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6
7    char *str = malloc(sizeof(char) * 13);
8
9    int message = 0;
10
11   getmsg(str, message);
12
13   if (!message)
14     free(str);
15     printf(message);
16
17 }
```



Clang Static Analyzer


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str);

  printf("%s", str);

  return 0;

}
```

Clang Static Analyzer


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 13);

  strcpy(str, "Hello World\n");

  free(str);

  printf("%s", str);

  return 0;

}
```

```
$ clang --analyze hello.c
hello.c:13:3:  warning:  Use of memory after it is freed
[unix.Malloc]
13 | printf("%s", str);
   | ^~~~~~~~~~~~~~~~~
1 warning generated.
$
```

Clang Static Analyzer


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 24);
  strcpy(str, "Hello World\n");
  char *str2 = malloc(sizeof(char) * 11);
  strcpy(str2, "I am here\n");

  make2_str(str, str2);

  printf("%s", str);

  return 0;
}
```

Clang Static Analyzer


```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

  char *str = malloc(sizeof(char) * 24);
  strcpy(str, "Hello World\n");
  char *str2 = malloc(sizeof(char) * 11);
  strcpy(str2, "I am here\n");

  make2_str(str, str2);

  printf("%s", str);

  return 0;
}
```

```
$ clang --analyze hello.c
$
```

Coccinelle (1)

Coccinelle (1)

- What it is:
  - Pattern matching tool
  - Uses *semantic patches*
  - Able to perform *transformations*

Coccinelle (1)

- What it is:
  - Pattern matching tool
  - Uses *semantic patches*
  - Able to perform *transformations*

```
1 @@
2 type t;
3 identifier x;
4 @@
5 t x;
6 ...
7 -free(x);
8 +pfree(x);
```

Coccinelle (1)

- What it is:
  - Pattern matching tool
  - Uses *semantic patches*
  - Able to perform *transformations*
- Limitations:
  - Scope
  - No aliasing
  - No symbolic execution

```
1 @@
2 type t;
3 identifier x;
4 @@
5 t x;
6 ...
7 -free(x);
8 +pfree(x);
```

Example of Strict script


```
1  @freeing@ forall@
2  type rt, t;
3  identifier f, i;
4  position p;
5
6  @@
7  rt f(..., t i, ...) {
8    ... when != if(...){ ... return ...;}
9    __FUNCTION__@p(..., i, ...)
10   <... when any
11   __FUNCTION__(..., i, ...)
12   ...>
13 }
```

Example of Dependent script


```
1  @freeing@ exists@
2  type rt, t1, t2;
3  identifier f, i, b;
4  position p;
5
6  @@
7  rt f(..., t1 i, ..., t2 b, ...) {
8    <+...
9    if (b) {... __FUNCTION__@p(..., i, ...) ...}
10   ...+>
11 }
```

Research Questions

Research questions

1. What **PostgreSQL-specific pitfalls** exist for developers of PostgreSQL extensions that can be **addressed through static program analysis**?
2. **Which approaches are most suitable** for tackling these problems? What are the benefits and limitations of the various methods?
3. How can existing tools be **extended to address PostgreSQL**-specific issues?